实验 3 函数与程序结构实验

3.1 实验目的

（1）熟悉和掌握函数的定义、声明；函数调用与参数传递，函数返回值类型的定义和返回值使用。

（2）熟悉和掌握不同存储类型变量的使用。

（3）练习使用集成开发环境中的调试功能：单步执行、设置断点、观察变量值。

3.2 实验内容

1．程序改错题

下面是计算 s=1!+2!+3!+…+n!的源程序(n<20)。在这个源程序中存在若干语法和逻辑错误。要求对该程序进行调试修改，使之能够输出如下结果：

k=1　　the sum is 1

k=2　　the sum is 3

k=3　　the sum is 9

……

k=20　　the sum is 2561327494111820313

```
/*实验 3-1 改错题程序：计算 s=1!+2!+3!+…+n!*/
#include <stdio.h>
int main(void)
{
        int k;
        for(k=1;k<=20;k++)
                printf("k=%d\tthe sum is %ld\n",k,sum_fac(k));
        return 0;
}
long sum_fac(int n)
{
        long s=0;
       int i,fac;
           for(i=1;i<=n;i++)
```

```
15                  fac*=i;
16              s+=fac;
17              return s;
18          }
```

解答：

错误修改：

1) sum_fac函数在main函数后面，这会导致隐式声明从而编译器报错 ，应当将sum_fac放在main函数前面
2) 第13行fac未定义且fac类型为int，后续计算会导致溢出，应当将fac改为long类型并且初始化，例如long fac = 1;
3) 第16行s+=fac;被放在for循环外，导致s只会最终与fac相等而不会是阶乘累加，应当用大括号包裹

2．程序修改替换题

（1）根据$\sum_{i=1}^{n} i! = \sum_{i=1}^{n-1} i! + n!$将实验 1 程序改错题中 sum_fac 函数修改为一个递归函数，用递归的方式计算$\sum_{i=1}^{n} i!$。

源代码清单：

```
/*实验 3-1 改错题程序：计算 s=1!+2!+3!+…+n!*/
#include <stdio.h>
long fact(int n)
{
    if (n <= 1)
        return 1;
    return n * fact(n - 1);
}

long sum_fac(int n)
{
    if (n <= 1)
        return 1;
    return sum_fac(n - 1) + fact(n);
}
```

```
int main(void)
{
    int k;
    for(k=1;k<=20;k++)
        printf("k=%d\tthe sum is %ld\n",k,sum_fac(k));
    return 0;
}
```

（2）下面是计算 $s = 1 + \frac{x}{1!} + \frac{x^2}{2!} + \frac{x^3}{3!} + L + \frac{x^n}{n!}$ 的源程序，其中 x 是浮点数，n 是整数。从键盘输入 x 和 n，然后计算 s 的值。修改该程序使之计算量最小，即减少程序中乘法和加法的运算次数，不是指减少代码的行数。

```
/*实验 3-2 程序修改替换第(2)题程序：根据公式计算 s*/
#include<stdio.h>
double mulx(double x,int n);
long fac(int n);
double sum(double x,int n)
{
    int i;
    double z=1.0;
    for(i=1;i<=n;i++)
    {
        z=z+mulx(x,i)/fac(i);
    }
    return z;
 }
 double mulx(double x,int n)
{
    int i;
    double z=1.0;
```

```
    for(i=0;i<n;i++)
    {
        z=z*x;
    }
    return z;
}
long fac(int n)
{
    int i;
    long h=1;
    for(i=2;i<=n;i++)
    {
        h=h*i;
    }
    return h;
}
int main()
{
    double x;
    int n;
    printf("Input x and n:");
    scanf("%lf%d",&x,&n);
    printf("The result is %lf:",sum(x,n));
    return 0;
}
```

源代码清单：

```
/*实验 3-2 程序修改替换第(2)题程序：根据公式计算 s*/
#include<stdio.h>
double sum(double x,int n)
{
    int i;
```

```
    double z=1.0;
    double term=1.0;
    for(i=1;i<=n;i++)
    {
        term *= x/i;
        z += term;
    }
    return z;
}
int main()
{
    double x;
    int n;
    printf("Input x and n:");
    scanf("%lf%d",&x,&n);
    printf("The result is %lf:",sum(x,n));
    return 0;
}
```

3．跟踪调试题

下面是计算 fabonacci 数列前 n 项和的源程序，现要求单步执行该程序，在 watch 窗口中观察 Ik,sum,n 值。具体操作如下：

（1） 设输入 5，观察刚执行完“scanf("%d",&k);”语句时，sum、k 的值是多少？

解答：

sum = 0，k = 5

（2） 在从 main 函数第一次进入 fabonacci 函数前的一刻，观察各变量的值是多少？返回后光条停留在哪个语句上？

解答：

i = 1, k = 5, sum = 0

停留在 printf("i=%d\tthe sum is %ld\n",i,sum);语句上

（3） 在从 main 函数第一次进入 fabonacci 函数后的一刻，观察光条从 main 函数“sum+=fabonacci(i);”语句调到了哪里？

解答：

调到了 if(n==1 || n==2)

（4） 在 fabonacci 函数内部单步执行，观察函数的递归执行过程。体会递归方式实现的计算过程是如何完成数计算的，并特别注意什么时刻结束递归，

然后直接从第一个 return 语句返回到了哪里？

解答：

当 i = 1 的时候结束递归

返回到了代码末尾

（5） 在 fabonacci 函数递归执行过程中观察参数 n 的变化情况，并回答为什么 k、sum 在 fabonacci 函数内部不可见？

解答：

n 会逐渐减 1，直到 n 等于 1 的时候 n 会变回最初传入的值（当 n = 1 或 2 时会直接返回）

因为 k 和 sum 是 main 函数的局部变量，作用域仅限于 main 内部。fabonacci 函数有自己独立的作用域，无法直接访问外部函数的局部变量。

/*实验 3-3 跟踪调试题程序：计算 fabonacci 数列前 n 项和*/

```
#include<stdio.h>
int main(void)
{
    int i,k;
    long sum=0,fabonacci(int n);
    printf("Inut n:");
    scanf("%d",&k);
    for(i=1;i<=k;i++){
        sum+=fabonacci(i);
        printf("i=%d\tthe sum is %ld\n",i,sum);
    }
    return 0;
}
long fabonacci(int n)
{
    if(n==1 || n==2)
        return 1;
```

```
    else
        return fabonacci(n-1)+fabonacci(n-2);
}
```

4．程序设计

（1）编程验证歌德巴赫猜想：任何一个大于等于 4 的偶数都是两个素数之和。要求设计一个函数，接受形参 n，以“n=n1+n2”的形式输出结果，若有多种分解情况，取 n1 最小的一个输出。

例如：n=6，输出“6=3+3”。

main 函数循环接收从键盘输入的整数 n，如果 n 是大于或等于 4 的偶数，调用上述函数进行验证。

1）解答思路：算法流程如图 3-1 所示：

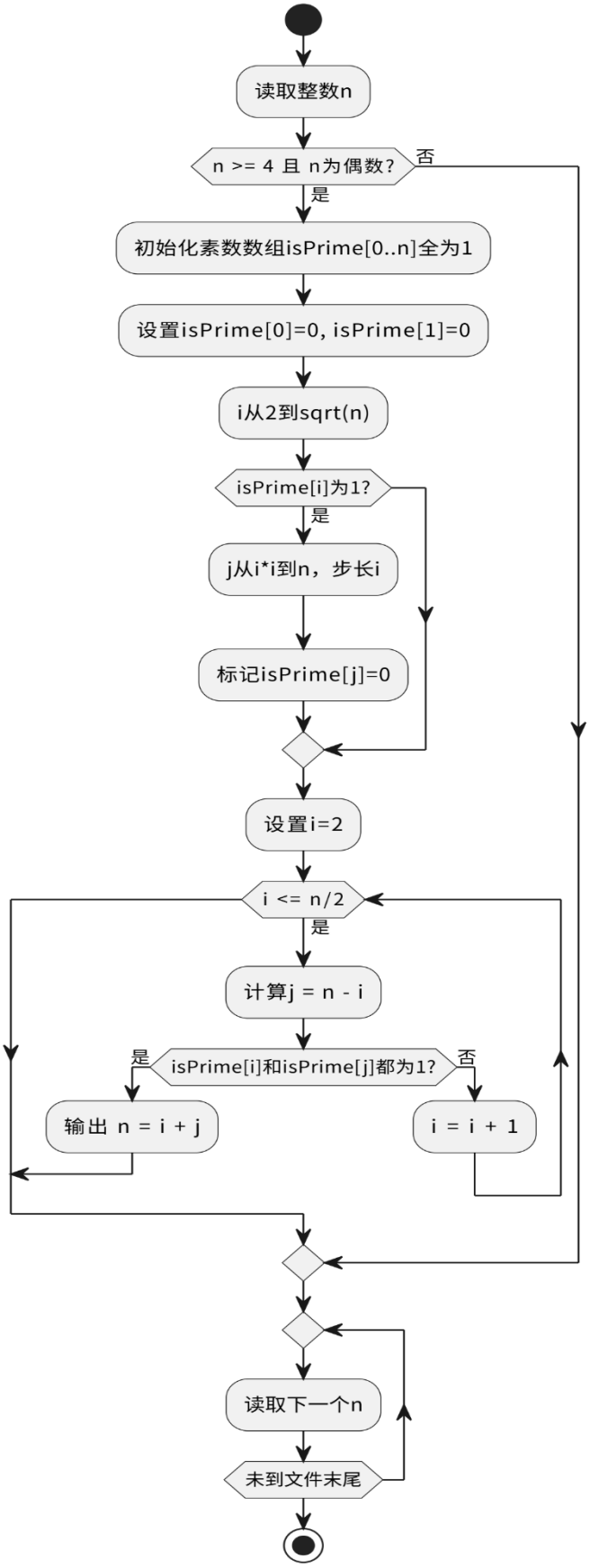

图 3-1 编程题 1 的程序流程图

2）源代码清单：

```
#include <stdio.h>
#include <stdlib.h>
int main(void) {
    int n;
    while (scanf("%d", &n) != EOF) {
        if (n >= 4 && n % 2 == 0) {
            int isPrime[n + 1];
            for (int i = 0; i <= n; i++) {
                isPrime[i] = 1;
            }
            isPrime[0] = isPrime[1] = 0;
            for (int i = 2; i * i <= n; i++) {
                if (isPrime[i]) {
                    for (int j = i * i; j <= n; j += i) {
                        isPrime[j] = 0;
                    }
                }
            }
            for (int i = 2; i <= n / 2; i++) {
                int j = n - i;
                if (isPrime[i] && isPrime[j]) {
                    printf("%d=%d+%d\n", n, i, j);
                    break;
                }
            }
        }
    }
}
```

3）测试

（a） 测试数据：

如 3-1 所示。

测试用例	程序输入	理论结果	运行结果
用例 1	6	3+3	3+3
用例 2	18	5+13	5+13
用例 3	10000	59+9941	59+9941

（b） 对应测试用例的运行结果如图 3-2 所示。

```
~/c-learn/experiment/3 ❯ cd "/home/locxl/c-learn/experiment/3/" && gcc 1_4_1.c -o 1_4_1 && "/home/locxl/c-learn/experiment/3/
"1_4_1
6
6=3+3
18
18=5+13
10000
10000=59+9941
^Z
[7]  + 224867 suspended  "/home/locxl/c-learn/experiment/3/"1_4_1
```

图 3-2 编程题 1 测试用例的运行结果

说明上述的运行结果与理论分析吻合，验证了程序的正确性。

（2）完全数（Perfect number），又称完美数或完备数，特点是它的所有真因子（即除了自身以外的约数，包括 1）之和恰好等于它本身。例如 6=1+2+3，28=1+2+4+7+14 等。

编程寻找 10000 以内的所有完全数。

要求设计一个函数，判定形参 n 是否为完全数，如果是，返回 1，否则返回 0。在 main 函数中调用该函数求 10000 以内的所有完全数，并以完全数的真因子之和的形式输出结果，例如“6=1+2+3”。程序输出中，每个完全数单独占一行。

解答：

1）解题思路：算法流程如图 3-3 所示。

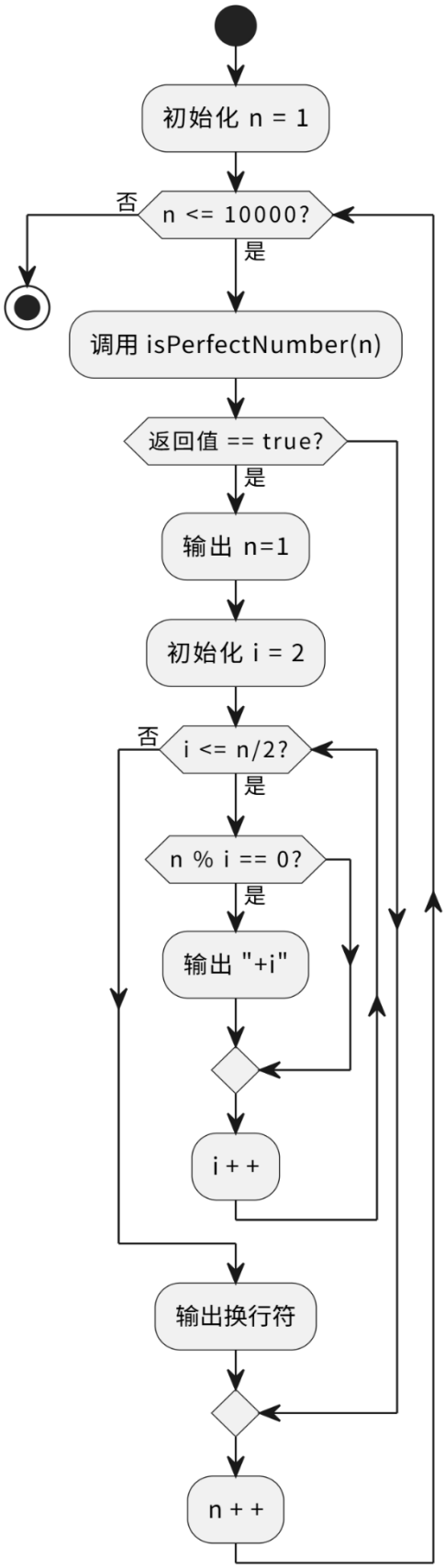

图 3-3 编程题 3 的程序流程图

2）源代码清单：

```
#include <stdio.h>

int isPerfectNumber(int n) {
	int sum = 0;
	for (int i = 1; i <= n / 2; i++) {
		if (n % i == 0) {
			sum += i;
		}
	}
	return sum == n;
}

int main(void) {
	int n;
	for (n = 1; n <= 10000; n++) {
		if (isPerfectNumber(n)) {
			printf("%d=1", n);
			for (int i = 2; i <= n / 2; i++) {
				if (n % i == 0) {
					printf("+%d", i);
				}
			}
			printf("\n");
		}
	}
	return 0;
}
```

3）测试

（a） 测试数据：

如表 3-2 所示。

表 3-2 编程题 2 的测试数据

测试用例	理论结果	实际结果
用例一	6=1+2+3 28=1+2+4+7+14 496=1+2+4+8+16+31+62+124+248 8128=1+2+4+8+16+32+64+127+254+508+1016+2032+4064	6=1+2+3 28=1+2+4+7+14 496=1+2+4+8+16+31+62+124+248 8128=1+2+4+8+16+32+64+127+254+508+1016+2032+4064

（b）对应测试测试用例的运行结果如图 3-4 所示。

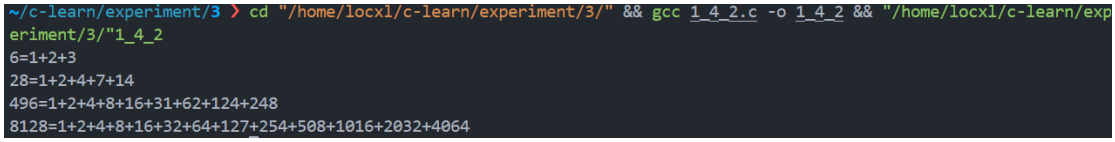

图 3-4 编程题 2 测试用例的运行结果

说明上述的运行结果与理论分析吻合，验证了程序的正确性。

（3）自幂数是指一个 n 位数，它的每个位上的数字的 n 次幂之和等于它本身。水仙花数是 3 位的自幂数，除此之外，还有 4 位的四叶玫瑰数、5 位的五角星数、6 位的六合数、7 位的北斗星数、8 位的八仙数等。

编写一个函数，判断其参数 n 是否为自幂数，如果是，返回 1；否则，返回 0。要求 main 函数能反复接收从键盘输入的整数 k，k 代表位数，然后调用上述函数求 k 位的自幂数，输出所有 k 位自幂数，并输出相应的信息，例如“3 位的水仙花数共有 4 个 153，370，371，407”。当 k=0 时程序结束执行。

解答：

1）解题思路：算法流程如图 3-5 所示。

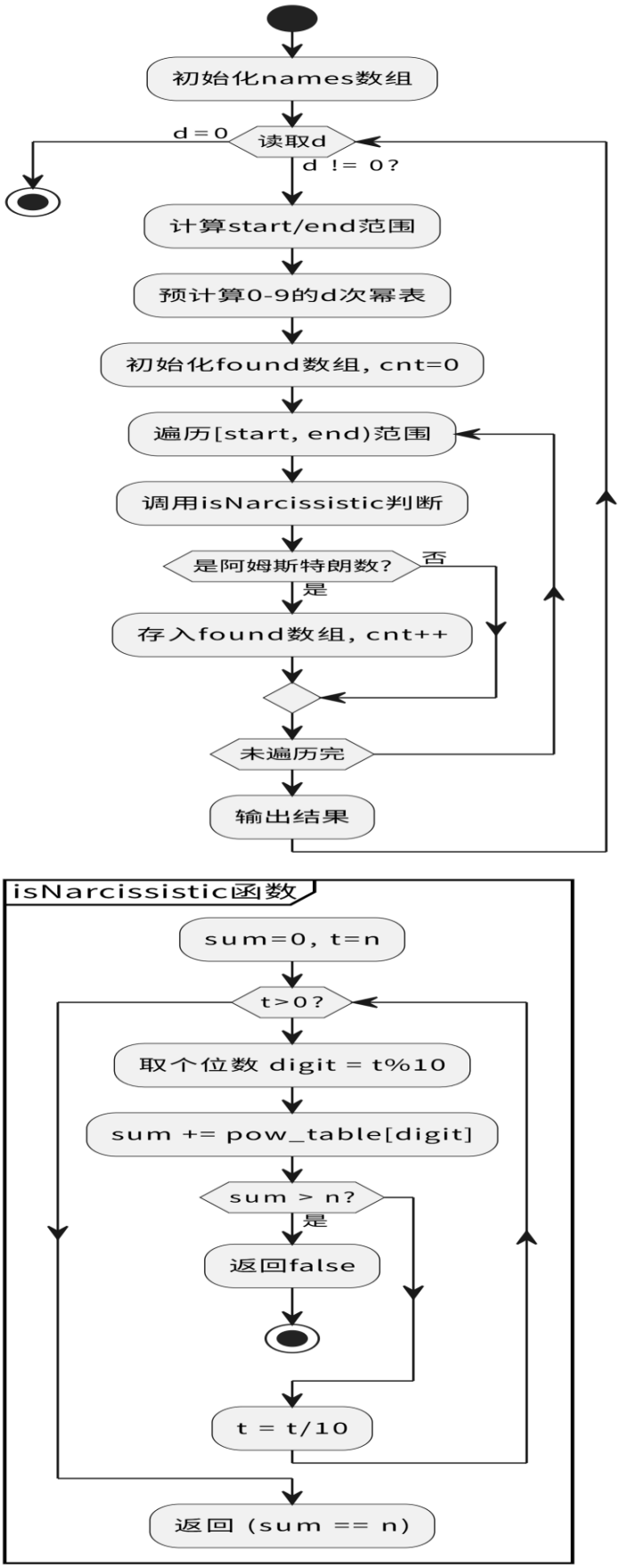

图 3-5 编程题 3 的程序流程图

2）源代码清单：

```
#include <stdio.h>

int isNarcissistic(int n, const int pow_table[]) {
    int sum = 0;
    int t = n;
    while (t > 0) {
        sum += pow_table[t % 10];
        if (sum > n) return 0;
        t /= 10;
    }
    return sum == n;
}

int main(void) {
    const char *names[9] = { NULL, NULL, NULL,
        "水仙花数",      // 3
        "四叶玫瑰数",    // 4
        "五角星数",      // 5
        "六合数",        // 6
        "北斗星数",      // 7
        "八仙数"          // 8
    };
    int d;
    while (scanf("%d", &d) == 1) {
        if (d == 0) break;

        int start = 1;
        for (int i = 1; i < d; i++) start *= 10;
        int end = start * 10;
```

```
        int pow_table[10];
        for (int digit = 0; digit < 10; digit++) {
            int p = 1;
            for (int k = 0; k < d; k++) p *= digit;
            pow_table[digit] = p;
        }

        int found[1024];
        int cnt = 0;
        for (int i = start; i < end; i++) {
            if (isNarcissistic(i, pow_table)) {
                if (cnt < 1024) found[cnt++] = i;
            }
        }

        printf("%d 位的%s 共有%d 个", d, names[d], cnt);
        if (cnt > 0) {
            for (int k = 0; k < cnt; k++) {
                if (k == 0) printf("%d", found[k]);
                else printf(",%d", found[k]);
            }
        }
        printf("\n");
    }
    return 0;
}
```

3）测试

（a） 测试数据：

如表 3-3 所示。

表 2-3 编程题 3 的测试数据

测试用例	程序输入	理论结果	运行结果
用例 1	3 4 5 6 7 8 9 0	3 位的水仙花数共有 4 个 153,370,371,407 4 位的四叶玫瑰数共有 3 个 1634,8208,9474 5 位的五角星数共有 3 个 54748,92727,93084 6 位的六合数共有 1 个 548834 7 位的北斗星数共有 4 个 1741725,4210818,9800817,9926315 8 位的八仙数共有 3 个 24678050,24678051,88593477 0	3 位的水仙花数共有 4 个 153,370,371,407 4 位的四叶玫瑰数共有 3 个 1634,8208,9474 5 位的五角星数共有 3 个 54748,92727,93084 6 位的六合数共有 1 个 548834 7 位的北斗星数共有 4 个 1741725,4210818,9800817,9926315 8 位的八仙数共有 3 个 24678050,24678051,88593477 0

（b） 对应测试测试用例的运行结果如图 3-6 所示。

图 3-6 编程题 3 测试用例的运行结果

说明上述的运行结果与理论分析吻合，验证了程序的正确性。

3.3 实验小结

通过本次实验，我遇到了一些问题，比如在程序改错题中，sum_fac 函数的隐式声明导致编译错误，需要将函数定义放在 main 函数之前；此外，变量 fac 未初始化且类型为 int，导致计算溢出，需将其改为 long 类型并初始化。在程序修改替换题中，通过优化算法减少了乘法和加法的运算次数，提高了程序效率。在跟踪调试题中，通过单步执行和观察变量值，深入理解了递归函数的执行过程和变量作用域的概念。这些经历让我更加熟悉了函数的定义、调用以及调试技巧，也增强了我对程序结构的理解和实际应用能力。